SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	DHANUSHREE S	**Reg. No.:**	192421218
Date:	21-08-2026	**Duration:**	60 minutes

Code of Conduct

*I **DHANUSHREE S/192421218** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Dhanushree S

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate

Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

Annexure A – CI/CD Pipeline Design Exercise

AI-BASED CROP DISEASE DETECTION AND FARMER ADVISORY PLATFORM

1. Introduction and Problem Statement Overview

1.1 Problem Statement Summary

The assigned problem statement is the design of an AI-Based Crop Disease Detection and Farmer Advisory Platform. The platform is envisioned as a multi-component software system that allows farmers to capture or upload images of crop leaves through a mobile application. A backend machine learning (ML) service analyses the uploaded image using a trained image-classification model to identify the presence and type of crop disease. Once a disease is identified, the platform generates an advisory in the farmer's local language, recommending suitable treatment, preventive measures, and, where relevant, connects the farmer with an agronomist through a web-based dashboard. The system therefore consists of four major components: a mobile application front end used by farmers, a backend REST API service, an ML model inference and training pipeline, and a web dashboard used by agronomists and administrators to monitor requests, review flagged cases, and manage advisory content.

Because the platform combines a conventional multi-tier web/mobile application with a continuously evolving machine learning model, its software delivery process must support two parallel delivery tracks: (a) the traditional application code (API, mobile app, dashboard) and (b) the ML model artifacts (training scripts, datasets, and trained model binaries). Both tracks must be integrated, tested, and released in a coordinated, automated, and auditable manner, which is precisely the problem that a CI/CD pipeline is designed to solve.

1.2 Purpose of the CI/CD Pipeline

The purpose of designing a Continuous Integration and Continuous Deployment (CI/CD) pipeline for this platform is to automate the entire journey of a code or model change, from the moment a developer commits new code to the moment that change is safely running in production and being observed. Manual build, test, and deployment processes are error-prone, slow, and difficult to audit, especially in a system that directly affects farmers' livelihoods through crop advisory decisions. An automated pipeline reduces the risk of shipping defective code or an inaccurate model, shortens the feedback loop for developers and data scientists, enforces consistent quality and security checks on every change, and provides a reliable, repeatable, and traceable mechanism for releasing new features or improved models.

Specifically, the CI/CD pipeline for this platform is expected to achieve the following objectives:

- Automatically build and validate every code change submitted to the repository, across the mobile application, backend API, and ML components.
- Run a comprehensive suite of automated tests (unit, integration, model-validation, security, and performance) before any change is allowed to progress toward production.
- Enforce code-quality and security gates so that only changes meeting defined thresholds are packaged and deployed.
- Package the application and the ML model into versioned, reproducible artifacts (container images and model registry entries).
- Deploy changes progressively through Development, Staging, and Production environments with appropriate approvals.
- Provide monitoring, alerting, and an automated rollback mechanism so that any regression, including a drop in model accuracy or an increase in error rate, can be detected and reversed quickly.

2. Requirement Analysis and CI/CD Needs

2.1 Functional Requirements of the Platform

Before designing the pipeline, it is necessary to understand what the platform itself must do, since these functional requirements shape what needs to be built, tested, and deployed on every release cycle.

- Farmers must be able to capture or upload leaf images through the mobile application, even under low-connectivity conditions in rural areas.
- The backend ML service must classify the uploaded image into a disease category (or 'healthy') and return a confidence score within an acceptable response time.
- The system must generate advisory content (treatment, dosage, preventive steps) in the farmer's preferred regional language.
- Agronomists must be able to review low-confidence predictions through a web dashboard and override or confirm the diagnosis.
- The platform must log every prediction request for later retraining, auditing, and model-performance analysis.
- Administrators must be able to trigger periodic retraining of the ML model as new labelled data becomes available.

2.2 Non-Functional Requirements

- Scalability: the inference API must handle seasonal spikes in usage (e.g., during sowing or pest-outbreak periods) without service degradation.
- Low latency: image classification results should typically be returned within a few seconds to remain usable in the field.
- Availability: the platform should target high availability, since farmers may depend on it for time-critical decisions.
- Data privacy and security: farmer location, contact, and crop data are sensitive and must be protected in transit and at rest.
- Maintainability: the codebase and ML pipeline must be easy to extend as new crops and diseases are added over time.
- Auditability: every deployment and model version must be traceable to the exact commit, dataset version, and test results that produced it.

2.3 CI/CD-Specific Requirements

Translating the functional and non-functional requirements above into CI/CD needs, the pipeline for this platform must satisfy the following requirements:

1. Multi-component orchestration: the pipeline must be capable of independently building and testing the mobile application, the backend API, the web dashboard, and the ML model, while still coordinating an integrated release when required.
2. Automated, gated quality checks: every change must pass unit tests, static code analysis, and security scanning before it can be merged or promoted.
3. Model-specific validation: in addition to conventional software tests, the pipeline must validate ML model quality, for example by checking classification accuracy, precision/recall per disease class, and inference latency against defined thresholds before a new model version is promoted.
4. Model and artifact versioning: every build must produce a uniquely versioned, immutable artifact (container image and/or model file) that can be traced back to its source commit and training dataset.
5. Progressive, environment-based delivery: changes must move through Development, Staging, and Production environments, with automated promotion between the earlier stages and a manual approval gate before Production.

6. Safe release strategies: because a faulty model can directly mislead farmers, the pipeline must support canary or blue-green release strategies for the inference service so that a new model or API version is only fully rolled out after it is verified against real or shadow traffic.
7. Rapid, automated rollback: if a regression is detected after deployment (elevated error rate, degraded model accuracy, or failed health checks), the pipeline must be able to roll back to the last known-good version automatically or with a single trigger.
8. Notifications and observability: the pipeline must notify the responsible team of failures at any stage, and the deployed system must be observable through logs, metrics, and dashboards.

2.4 Stakeholders and Team Structure

The primary stakeholders involved in the CI/CD process include the mobile and backend developers who contribute application code, the data science team responsible for training and validating the disease-classification model, the DevOps/platform engineer who maintains the pipeline and infrastructure, the QA team responsible for test strategy and sign-off, and the agronomist/domain-expert team that validates advisory content accuracy. The pipeline is designed so that each stakeholder's contribution is automatically validated and integrated without requiring manual coordination for every release.

3. CI/CD Pipeline Architecture and Workflow

3.1 High-Level Pipeline Architecture

The pipeline architecture designed for the AI-Based Crop Disease Detection and Farmer Advisory Platform follows a linear, stage-gated workflow, beginning at source-code commit and ending in a monitored production deployment, with an automated rollback path connected to the production stage. The full architecture diagram is presented below, followed by a detailed explanation of every stage.

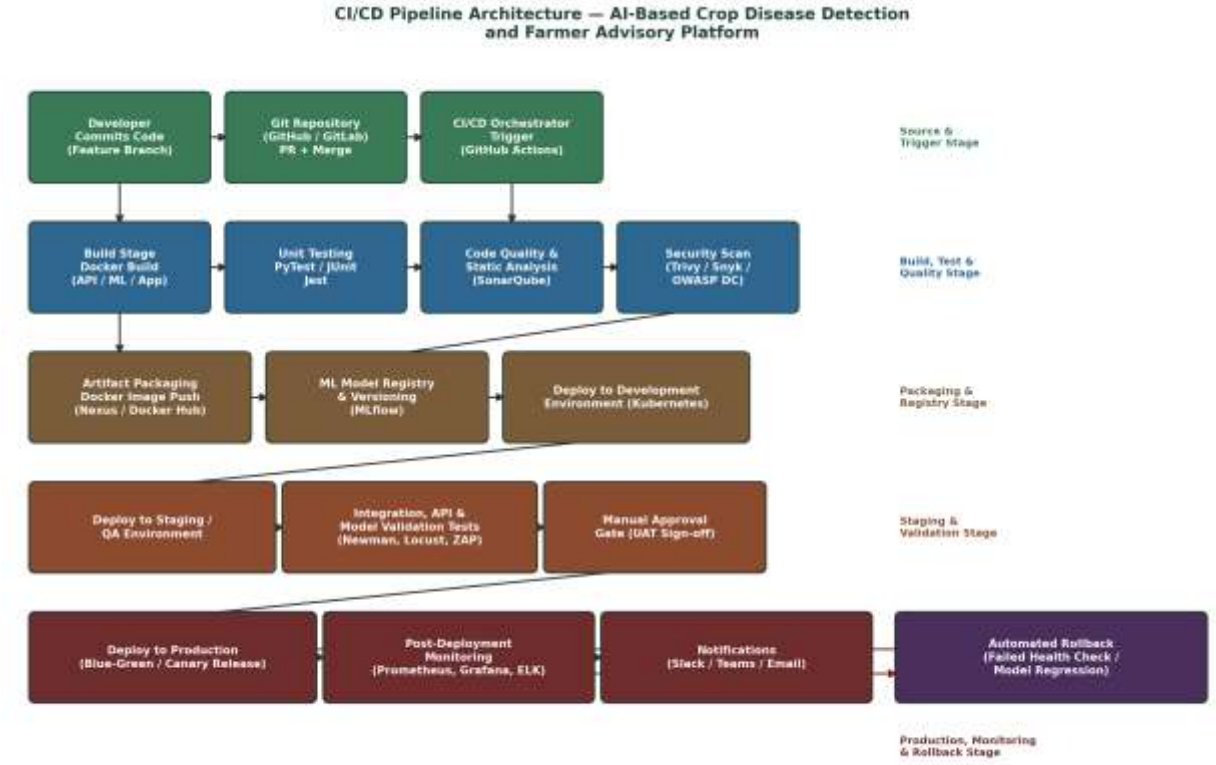

Figure 1: CI/CD Pipeline Architecture Diagram for the Crop Disease Detection and Farmer Advisory Platform

3.2 Stage-by-Stage Explanation of the Pipeline

Stage 1 – Source Code Management and Trigger: Developers work on isolated feature branches and raise a pull request against the develop branch. A merge, or a direct push depending on branch protection rules, automatically triggers the CI/CD orchestrator (GitHub Actions in this design). Both application code and ML training/inference code are version-controlled in the same monorepo-style repository structure, organised into clearly separated directories for the mobile app, backend API, ML service, and infrastructure-as-code, so that the orchestrator can selectively trigger only the relevant sub-pipeline for a given change.

Stage 2 – Build: The orchestrator checks out the code and builds the affected component. The backend API and ML inference service are containerised using Docker, while the mobile application is compiled/bundled using its native build toolchain (Gradle for Android, Xcode build tools for iOS) and the web dashboard is built using its front-end build tool. A failed build immediately stops the pipeline and notifies the responsible developer.

Stage 3 – Unit Testing: Once the build succeeds, an automated unit-test suite executes against the backend API (PyTest), the ML data-processing and inference functions (PyTest), and the mobile/dashboard front-end (Jest). Unit tests validate individual functions and modules in isolation, using mocked dependencies, and must pass a minimum coverage threshold before the pipeline proceeds.

Stage 4 – Code Quality and Static Analysis: The code is analysed using SonarQube (supplemented by ESLint for JavaScript/TypeScript and Pylint for Python) to detect code smells, duplication, maintainability issues, and rule violations. A quality gate defined in SonarQube (for example, zero new critical/blocker issues and a minimum maintainability rating) must pass for the pipeline to continue.

Stage 5 – Security Scanning: Dependency and container images are scanned for known vulnerabilities using Trivy for container images and OWASP Dependency-Check / Snyk for application dependencies. Any critical or high-severity vulnerability without an approved exception fails the pipeline at this stage, preventing insecure artifacts from being packaged.

Stage 6 – Packaging and Artifact/Model Registry: Once all quality and security gates are satisfied, the pipeline builds a versioned, immutable Docker image for the API and ML inference service and pushes it to a container registry (Nexus/Docker Hub), tagged with the Git commit hash and semantic version. In parallel, if the change relates to the ML model, the newly trained model artifact is registered in MLflow's model registry, along with its evaluation metrics, training dataset version, and lineage information, allowing every deployed model to be traced back to the data and code that produced it.

Stage 7 – Deployment to Development Environment: The packaged artifact is automatically deployed to a Kubernetes-based development environment, where it is exposed only to the internal engineering team for exploratory verification and early integration testing.

Stage 8 – Deployment to Staging/QA and Validation Testing: The artifact is promoted to a staging environment that mirrors production as closely as possible. Here, integration tests, API contract tests, model-validation tests, performance/load tests, and dynamic security tests are executed automatically against the staging deployment.

Stage 9 – Manual Approval Gate: Given the domain sensitivity of agricultural advisory content, a manual approval step is enforced before production release. A designated reviewer (release manager, QA lead, or, for model changes, a data-science lead) inspects the staging test results and either approves or rejects the promotion to production.

Stage 10 – Deployment to Production: Upon approval, the pipeline deploys the artifact to production using a blue-green strategy for the API/dashboard and a canary strategy for the ML inference service, so that a new model version first serves a small percentage of real traffic before being fully rolled out.

Stage 11 – Post-Deployment Monitoring and Notifications: Once live, the system is continuously monitored using Prometheus and Grafana for infrastructure/application metrics and the ELK stack for centralized logging. Notifications for every pipeline failure, security finding, or production alert are sent automatically to the responsible team via Slack/Microsoft Teams and email.

Stage 12 – Automated Rollback: If monitoring detects a failed health check, an elevated error rate, or a statistically significant drop in model prediction confidence/accuracy after a release, the pipeline automatically reroutes traffic back to the previous stable version (for blue-green deployments) or halts the canary rollout and reverts to the last known-good model version registered in MLflow.

3.3 Branching Strategy

The pipeline is designed around a simplified GitFlow-style branching model suited to a small, cross-functional team. The main branch always reflects the current production state; the develop branch integrates completed features and is continuously deployed to the Development environment; short-lived feature/branchname branches are created for individual features or bug fixes and merged into develop via pull request only after passing all automated CI checks; and release branches are cut from develop for staging validation before being merged into main and tagged with a semantic version number, which triggers the production deployment stage.

3.4 Pipeline Triggers

- Pull request trigger: runs build, unit tests, code quality, and security scanning on every pull request to provide fast feedback before merge.

- Push/merge trigger: a merge into develop triggers the full pipeline up to the Development/Staging deployment.
- Tag-based release trigger: pushing a semantic version tag on main triggers packaging and the production deployment sequence, including the manual approval gate.
- Scheduled retraining trigger: a scheduled (e.g., weekly) job re-trains the ML model on newly labelled data accumulated from farmer submissions and agronomist corrections, automatically feeding the resulting model into the same validation and registry stages as a code-driven change.

4. Tools and Technology Selection

The table below summarizes the tools selected for each stage of the CI/CD pipeline, along with the justification for each choice. The selection prioritizes tools that are open-source or widely available, integrate well with a Kubernetes/Docker-based deployment target, and collectively provide end-to-end coverage of build, test, security, deployment, and observability concerns for both the conventional application code and the machine learning components of the platform.

Pipeline Stage	Tool(s) Selected	Justification
Source Code Management	GitHub / GitLab	Industry-standard hosting with built-in pull-request workflows, branch protection rules, and native integration with CI/CD orchestrators, enabling fine-grained access control for a multi-team project.
CI/CD Orchestration	GitHub Actions (Jenkins as alternative)	Tightly integrated with the source repository, YAML-based pipeline-as-code, large marketplace of pre-built actions for Docker, Kubernetes, and testing tools, and no separate server to maintain, reducing operational overhead for a small team.
Build & Containerization	Docker	Provides reproducible, portable build artifacts across development, staging, and production, eliminating environment-inconsistency issues, and is a prerequisite for Kubernetes-based deployment.
Backend Unit Testing	PyTest	Widely adopted, low-boilerplate Python testing framework with strong fixture and parameterization

		support, well suited to both the Flask/FastAPI backend and the ML inference/data-processing code.
Frontend/Mobile Testing	Jest, Espresso/XCTest	Jest provides fast, snapshot-capable testing for the web dashboard's JavaScript/React code; Espresso and XCTest are the native frameworks for reliable Android and iOS UI/unit testing respectively.
Code Quality Analysis	SonarQube, ESLint, Pylint	SonarQube offers a unified quality gate across multiple languages (Python, JavaScript, Java/Kotlin, Swift) with historical trend tracking; ESLint and Pylint provide fast, language-specific linting during local development and CI.
Security Scanning	Trivy, OWASP Dependency-Check / Snyk	Trivy scans container images for OS and library vulnerabilities; OWASP Dependency-Check/Snyk scans application dependencies for known CVEs, together covering both the infrastructure and application security surface.
Artifact/Container Registry	Docker Hub / Nexus Repository	Provides versioned, access-controlled storage for built container images, with tagging support that enables traceability back to the exact commit that produced each image.
ML Model Registry & Versioning	MLflow	Purpose-built for tracking ML experiments, model versions, evaluation metrics, and lineage, enabling controlled promotion of models between staging and production and rapid rollback to a previous model version.
Container Orchestration	Kubernetes with Helm	Enables declarative, scalable deployment of the API, dashboard, and inference service across Development, Staging, and Production, with Helm charts standardizing configuration across

		environments and supporting blue-green/canary rollout patterns.
Infrastructure as Code	Terraform	Allows the cloud infrastructure (Kubernetes clusters, networking, storage, managed database) to be version-controlled and reproducibly provisioned across environments, reducing configuration drift.
API/Integration Testing	Postman / Newman	Newman allows Postman API test collections to be executed headlessly inside the CI pipeline, validating request/response contracts for the backend and inference APIs on every staging deployment.
Performance/Load Testing	Locust / Apache JMeter	Simulates concurrent farmer requests to the image-classification API to validate latency and throughput under expected and peak seasonal load before production release.
Dynamic Security Testing	OWASP ZAP	Performs automated dynamic application security testing (DAST) against the running staging environment to catch runtime vulnerabilities not visible through static analysis alone.
Secrets Management	HashiCorp Vault	Centralizes and encrypts sensitive credentials (database passwords, API keys, model-registry tokens) instead of storing them in pipeline configuration files, reducing credential-leakage risk.
Monitoring & Observability	Prometheus, Grafana	Prometheus collects real-time metrics (latency, error rate, resource usage, model-confidence distribution) and Grafana visualizes them on dashboards used for both operational monitoring and rollback decisions.
Centralized Logging	ELK Stack (Elasticsearch, Logstash, Kibana)	Aggregates logs from the API, inference service, and mobile backend into a searchable central store,

		essential for diagnosing production issues and for auditing advisory decisions.
Notifications	Slack / Microsoft Teams, Email (SMTP)	Provides immediate, low-friction alerts to the responsible team for pipeline failures, security findings, and production incidents, integrated directly with the CI/CD orchestrator and monitoring stack.

Together, these tools form a cohesive toolchain in which each stage's output feeds directly into the next: source commits trigger builds, builds produce artifacts that are tested and scanned, verified artifacts are registered and versioned, and versioned artifacts are deployed and observed, with feedback from observability flowing back into the retraining and rollback stages of the pipeline.

5. Automated Testing Strategy

Automated testing is embedded at multiple points in the pipeline so that defects, regressions, and model-quality issues are caught as early and as cheaply as possible. The strategy combines conventional software testing techniques with ML-specific validation, reflecting the hybrid nature of the platform.

5.1 Unit Testing

Every function and module in the backend API and ML data-processing code is covered by PyTest unit tests, using mocked database and external-service calls to keep tests fast and isolated. The mobile application and web dashboard use Jest (JavaScript/React) and native frameworks (Espresso for Android, XCTest for iOS) for component-level unit tests. A minimum line-coverage threshold (for example, 80 percent) is enforced as a quality gate; the pipeline fails if a change reduces coverage below this threshold.

5.2 Machine Learning Model Testing

Because the ML model directly influences farmer decisions, it is subjected to a dedicated validation suite beyond standard unit tests, including:

- Data validation tests that check incoming training data for schema consistency, missing labels, and class imbalance before training begins.
- Accuracy and per-class metric tests that compare the newly trained model's accuracy, precision, recall, and F1-score per disease category against a held-out validation set and a minimum acceptable threshold.

- Regression tests that compare the new model's performance against the currently deployed model on a fixed benchmark set, blocking promotion if the new model performs worse on any critical disease class.
- Bias and fairness checks that evaluate model performance across different crop varieties, image-capture conditions, and regions to avoid systematically poor advisories for specific farmer groups.
- Inference latency tests that measure prediction response time to confirm it remains within the acceptable range for field usage.

5.3 Integration Testing

Integration tests validate that the mobile app, backend API, ML inference service, and database interact correctly, exercising real (non-mocked) calls between components inside the staging environment after deployment.

5.4 API Contract Testing

Postman collections, executed headlessly through Newman inside the pipeline, validate that every API endpoint (image upload, prediction retrieval, advisory lookup, agronomist review) returns the expected response schema, status codes, and error handling, protecting mobile and dashboard clients from breaking backend changes.

5.5 UI and Mobile Automated Testing

Automated UI test suites (Espresso for Android, XCTest for iOS, and Selenium/Cypress-style tests for the web dashboard) validate key user journeys such as image capture and upload, advisory display, and agronomist review, running against the staging deployment on every release candidate.

5.6 Performance and Load Testing

Locust or Apache JMeter scripts simulate concurrent image-upload and prediction requests to measure the inference API's throughput and latency under expected and peak seasonal load (for example, during a regional pest outbreak), ensuring the service can scale appropriately before production release.

5.7 Security Testing

In addition to static dependency and image scanning earlier in the pipeline, OWASP ZAP performs automated dynamic application security testing against the running staging environment, probing for common vulnerabilities such as injection flaws, insecure authentication, and misconfigured endpoints.

5.8 Test Coverage and Quality Gate Thresholds

The pipeline enforces the following representative quality gates, any failure of which halts promotion to the next stage: minimum 80 percent unit-test coverage; zero critical or blocker SonarQube issues; zero unresolved critical/high-severity vulnerabilities; ML model accuracy not lower than the currently deployed model on the benchmark set; and all API contract and integration tests passing with zero failures.

6. Deployment Strategy

6.1 Environment Strategy

The platform uses three progressively production-like environments. The Development environment receives every successful build from the develop branch and is used for internal exploratory testing by engineers and data scientists. The Staging/QA environment mirrors production configuration (same Kubernetes cluster type, comparable data volumes, and identical service topology) and is where the full automated validation suite (integration, contract, performance, and security tests) and the manual UAT sign-off take place. The Production environment serves real farmers and agronomists and is only reachable through the approval-gated release process.

6.2 Deployment Approach

Blue-Green Deployment for the API and Dashboard: Two identical production environments (blue and green) are maintained. The new version is deployed to the idle environment and verified using automated smoke tests; traffic is then switched at the load balancer/ingress level once verification succeeds. If a problem is detected, traffic is switched back to the previous environment almost instantly, minimizing downtime and risk.

Canary Release for the ML Inference Service: Because a faulty model can silently produce poor agricultural advice rather than an obvious application error, new model versions are rolled out gradually. Initially, a small percentage of prediction traffic (for example, 5 percent) is routed to the new model version while the majority continues to use the current stable model. Prediction confidence, agronomist override rate, and error metrics are monitored for the canary slice before traffic is progressively increased to 25 percent, 50 percent, and finally 100 percent.

6.3 Infrastructure as Code

All environment infrastructure, including Kubernetes namespaces, node pools, networking rules, managed database instances, and storage buckets for uploaded images, is defined using Terraform scripts stored in version control. This ensures that Development, Staging, and Production environments are provisioned consistently, changes to infrastructure go through the same pull-request review process as application code, and environments can be rebuilt reproducibly if needed.

6.4 Configuration Management

Environment-specific configuration (database connection strings, feature flags, model-registry endpoints) is externalized using Kubernetes ConfigMaps for non-sensitive values and HashiCorp Vault-backed Kubernetes Secrets for sensitive values, ensuring the same container image can be promoted unchanged from Staging to Production, with only its configuration differing.

6.5 Approval Workflow

Promotion from Development to Staging is fully automated once all CI quality gates pass. Promotion from Staging to Production requires an explicit manual approval step within the CI/CD orchestrator, recorded with the approver's identity and timestamp, after the release manager or data-science lead reviews the staging test report, security scan summary, and (for model changes) the model-validation report.

7. Security, Quality Gates, Notifications, and Rollback Mechanisms

7.1 Security Practices Across the Pipeline

- Secrets management: all credentials, API keys, and tokens are stored in HashiCorp Vault and injected into the pipeline and runtime environment at execution time, never committed to source control.

- Image and dependency scanning: every container image is scanned with Trivy and every application dependency with OWASP Dependency-Check/Snyk before packaging, blocking artifacts with unresolved critical vulnerabilities.
- Role-based access control (RBAC): access to the Kubernetes clusters, artifact registry, and model registry is restricted by role, so that only authorized pipeline service accounts and senior engineers can trigger production deployments.

- Encryption in transit and at rest: all client-server communication (mobile app, dashboard, API) uses HTTPS/TLS, and farmer data stored in the database and object storage is encrypted at rest.
- Data privacy compliance: farmer personal data (name, contact number, location) is minimized, access-controlled, and handled in line with applicable data-protection requirements, with uploaded images and derived data retained only as long as necessary for advisory and retraining purposes.
- Signed, immutable artifacts: container images are tagged with immutable commit-based versions, preventing tampering between the build stage and deployment.

7.2 Quality Gates

Quality gates act as automated checkpoints that a change must pass before advancing to the next pipeline stage. In this design, gates are enforced after unit testing (coverage threshold), after static analysis (SonarQube quality gate: no new critical/blocker issues), after security scanning (no unresolved critical/high vulnerabilities), after staging validation (all integration, contract, and performance tests passing), and after ML model validation (accuracy and fairness thresholds met, no regression versus the current production model). A failure at any gate stops the pipeline and prevents the change from progressing further.

7.3 Notification Mechanism

The pipeline integrates with Slack and Microsoft Teams through webhook notifications, and with email via SMTP, to alert the relevant team immediately when a build fails, a quality or security gate is not met, a manual approval is pending, a production deployment completes, or a production alert is triggered by the monitoring stack. Notifications are routed to different channels depending on severity, for example a dedicated high-priority channel for production incidents versus a general channel for routine build status.

7.4 Rollback Strategy

The rollback strategy is designed around fast, low-risk reversal rather than manual investigation under pressure.

- Application rollback: because Blue-Green deployment keeps the previous production version running in the idle environment, rolling back the API/dashboard is a near-instantaneous traffic switch back to the previous environment, triggered automatically if post-deployment smoke tests or health checks fail, or manually by an on-call engineer.

- Model rollback: if the canary release of a new model shows degraded confidence, higher agronomist-override rates, or breaches a defined error-rate threshold, the pipeline automatically halts traffic shifting and reverts the inference service to the last known-good model version recorded in the MLflow model registry.
- Database/schema rollback: database migrations are written to be backward-compatible where possible, and each migration is paired with a corresponding down-migration script so that a rollback of application code does not leave the database in an incompatible state.
- Post-rollback review: every rollback event automatically generates an incident record and notification, ensuring the root cause is investigated before the change is reattempted.

7.5 Monitoring and Observability

Prometheus continuously collects metrics such as request latency, error rate, resource utilization, and model-prediction confidence distribution, visualized through Grafana dashboards accessible to the engineering and data-science teams. The ELK stack aggregates application and infrastructure logs for troubleshooting and auditing. Alert rules defined on top of these metrics (for example, error rate exceeding a defined percentage over a five-minute window, or a sustained drop in average model confidence) automatically trigger the notification and rollback mechanisms described above.

8. Pipeline Diagram and End-to-End Workflow Explanation

The figure below repeats the pipeline architecture diagram to accompany the end-to-end walkthrough that follows, tracing a single change from the moment it is committed to the moment it is fully live and monitored in production.

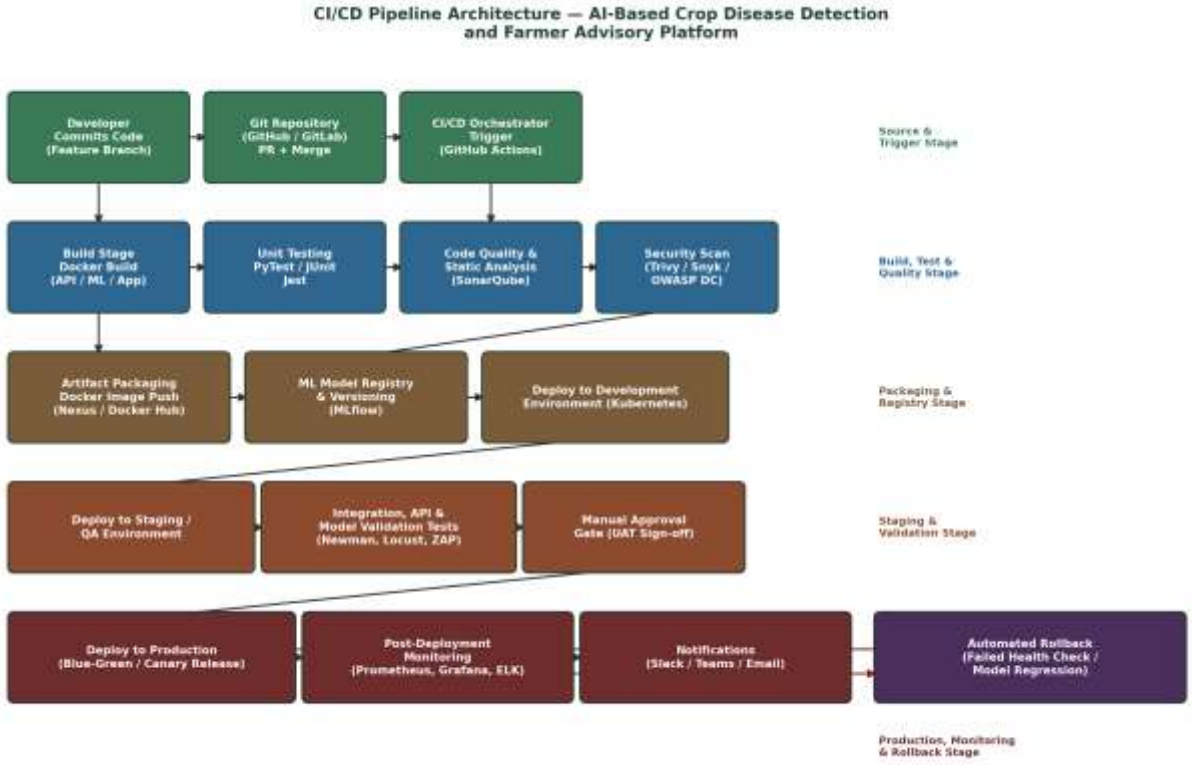

Figure 2: End-to-End CI/CD Workflow, from Code Commit to Production Deployment

8.1 Walkthrough: From Commit to Deployment

- A developer or data scientist commits a change (application code or a retrained model) to a feature branch and opens a pull request against the develop branch.
- The CI/CD orchestrator automatically triggers the pipeline, checking out the code and building the affected component(s) using Docker.
- Unit tests execute against the changed component; a failure at this point stops the pipeline immediately and notifies the developer through Slack/email, with the pull request marked as failing.
- SonarQube performs static code analysis and evaluates the change against the configured quality gate; ESLint/Pylint provide fast, in-pipeline linting feedback.
- Trivy and OWASP Dependency-Check/Snyk scan the built container image and its dependencies for known vulnerabilities.
- If all checks pass, the pull request is approved and merged into develop. This merge triggers packaging: a versioned Docker image is pushed to the artifact registry, and, for model changes, the new model is registered in MLflow with its evaluation metrics.

- The pipeline automatically deploys the new artifact to the Development environment for exploratory verification.
- On creation of a release branch/tag, the artifact is promoted to Staging, where integration tests, Postman/Newman API contract tests, Locust/JMeter performance tests, OWASP ZAP dynamic security tests, and (for model changes) the full ML validation suite execute automatically.
- A designated reviewer inspects the consolidated staging test report and either approves or rejects promotion to Production through the pipeline's manual approval gate.
- Upon approval, the pipeline deploys to Production using Blue-Green deployment for the API/dashboard and a Canary rollout for the ML inference service, gradually increasing traffic to the new model version while monitoring key metrics.
- Prometheus, Grafana, and the ELK stack continuously observe the production deployment; if any monitored metric breaches its threshold, the automated rollback mechanism reverts the affected component to its last known-good version and raises an incident notification.
- Once the release is confirmed stable, it becomes the new baseline against which the next canary rollout or blue-green switch will be compared, and the cycle repeats for the next change.

8.2 Traceability

At every stage of this workflow, the pipeline preserves traceability: the deployed container image tag maps to a specific Git commit hash, the deployed model version maps to a specific MLflow run with its training dataset and evaluation metrics, and every promotion and rollback event is logged with a timestamp and responsible actor. This traceability is essential both for debugging production issues and for demonstrating regulatory and academic accountability for the advisory decisions the platform makes.

9. Conclusion

The CI/CD pipeline designed in this document addresses the specific challenges posed by the AI-Based Crop Disease Detection and Farmer Advisory Platform: a hybrid system combining conventional mobile/web/API development with a continuously evolving machine learning model that directly influences agricultural decisions made by farmers. By automating source-code integration, multi-layered testing (unit, integration, contract, performance, security, and ML-specific validation), static and dynamic security scanning, versioned artifact and model registry management, progressive Blue-Green and Canary deployment strategies, and automated rollback triggered by real-time monitoring, the pipeline substantially reduces the risk of shipping defective code or an inaccurate model to production.

The design also embeds quality gates and a manual approval checkpoint before production release, balancing the speed benefits of automation with the domain-specific need for human oversight in agricultural advisory content. Overall, this pipeline provides a repeatable, auditable, and secure path from every code or model change to a reliably running production system, directly fulfilling the CO4 objective of implementing robust testing, quality assurance, and DevOps practices with an emphasis on security, reliability, and ethics.

10. References

1. Humble, J., & Farley, D., Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley.

2. Kim, G., Humble, J., Debois, P., & Willis, J., The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations, IT Revolution Press.

3. Sculley, D. et al., Hidden Technical Debt in Machine Learning Systems, Advances in Neural Information Processing Systems (NeurIPS).

4. Official documentation: GitHub Actions – docs.github.com/actions.

5. Official documentation: SonarQube – docs.sonarsource.com.

6. Official documentation: Docker – docs.docker.com.

7. Official documentation: Kubernetes – kubernetes.io/docs.

8. Official documentation: MLflow – mlflow.org/docs.

9. Official documentation: Prometheus – prometheus.io/docs and Grafana – grafana.com/docs.

10. OWASP Foundation – OWASP Dependency-Check and OWASP ZAP documentation, owasp.org.

11. HashiCorp Vault and Terraform documentat